

2.5. Modelos funcionales (Perspectiva funcional)

2.5.1. Concepto y propósito de los modelos funcionales

La perspectiva funcional representa el comportamiento esperado del sistema desde el punto de vista de las funciones y servicios que deberá proporcionar para satisfacer las necesidades de los interesados. Mientras que el modelo de datos describe qué información administra el sistema, el modelo funcional explica cómo dicha información es capturada, procesada, transformada y distribuida entre los diferentes actores y componentes involucrados. En consecuencia, ambos modelos constituyen perspectivas complementarias dentro del proceso de Ingeniería de Requisitos y permiten obtener una representación más completa del dominio del problema [1, 2].

IEEE/ISO/IEC 29148 establece que la especificación de requisitos puede apoyarse en diferentes modelos de representación para mejorar la comprensión del sistema y reducir ambigüedades durante el análisis. Entre estos modelos se encuentran aquellos orientados a describir funciones, flujos de información, escenarios de interacción y comportamiento esperado del sistema, los cuales complementan la especificación textual del Documento de Especificación de Requisitos de Software (SRS) [1]. De forma similar, SWEBOK considera que el análisis funcional constituye una actividad esencial para transformar las necesidades identificadas durante la elicitación en requisitos estructurados y verificables, favoreciendo la comunicación entre usuarios, analistas y desarrolladores [2].

Desde una perspectiva metodológica, los modelos funcionales buscan responder principalmente a las siguientes preguntas:

- ¿Qué funciones debe realizar el sistema?
- ¿Cómo fluye la información entre los distintos procesos?
- ¿Qué actores interactúan con el sistema?
- ¿Qué resultados produce cada proceso?

Responder estas interrogantes permite identificar responsabilidades, dependencias funcionales y posibles inconsistencias antes de iniciar el diseño arquitectónico o la implementación del software. Asimismo, facilita la validación temprana de los requisitos con las partes interesadas, ya que los modelos funcionales suelen resultar más intuitivos que una especificación completamente textual.

En el proyecto SIGA, por ejemplo, los modelos funcionales permiten representar el flujo completo de información desde la captura de datos mediante sensores IoT hasta la generación de alertas, el control remoto de dispositivos tecnológicos y la creación de solicitudes de mantenimiento. De esta manera, requisitos funcionales como RF-01 (Recopilación de datos ambientales), RF-04 (Monitoreo y control remoto de proyectores), RF-08

(Generación y envío de alertas por anomalías) y RF-12 (Gestión de solicitudes de mantenimiento) pueden analizarse como procesos relacionados que intercambian información de forma continua.

2.5.2. Diagramas de Flujo de Datos (DFD)

Entre los modelos funcionales clásicos utilizados durante el análisis de requisitos destacan los Diagramas de Flujo de Datos (Data Flow Diagrams, DFD), cuya finalidad consiste en representar gráficamente el movimiento de la información entre procesos, entidades externas y almacenes de datos. A diferencia de otros modelos centrados en la secuencia temporal de ejecución, el DFD enfatiza la transformación de datos realizada por cada proceso y la interacción existente entre los distintos componentes del sistema [3].

Según Maciaszek, los Diagramas de Flujo de Datos constituyen una herramienta especialmente útil durante las primeras etapas del análisis, ya que permiten describir el comportamiento funcional del sistema sin depender de detalles relacionados con la arquitectura de software o las tecnologías de implementación. En consecuencia, favorecen el análisis del problema antes de abordar decisiones de diseño [3].

Un DFD se encuentra conformado por cuatro elementos fundamentales:

- **Procesos:** representan las actividades que transforman datos de entrada en información de salida.
- **Flujos de datos:** describen el movimiento de información entre procesos, entidades externas y almacenes de datos.
- **Almacenes de datos:** representan repositorios donde la información es almacenada para su posterior utilización.
- **Entidades externas:** corresponden a usuarios u otros sistemas que intercambian información con el sistema analizado.

Estos elementos permiten construir una representación funcional independiente de aspectos tecnológicos, facilitando la comunicación con usuarios finales y responsables del negocio durante las actividades de validación de requisitos.

2.5.3. Niveles de abstracción de los DFD

Una de las principales características de los Diagramas de Flujo de Datos es su naturaleza jerárquica. El modelo comienza con una representación de alto nivel denominada Diagrama de Contexto o Nivel 0, en la cual el sistema completo es tratado como un único proceso que interactúa con entidades externas. Posteriormente, este proceso puede descomponerse en procesos de menor nivel, obteniendo Diagramas de Nivel 1, Nivel 2 y niveles sucesivos según la complejidad del sistema [3].

Esta descomposición gradual permite administrar la complejidad del análisis y facilita la comprensión del funcionamiento general del sistema antes de estudiar el detalle de cada proceso individual. Asimismo, contribuye a mantener la consistencia entre los distintos niveles de especificación, principio que resulta coherente con el refinamiento progresivo promovido por IEEE/ISO/IEC 29148 para la ingeniería de requisitos [1].

En el caso del sistema SIGA, el Diagrama de Contexto representará la interacción entre el sistema y actores como el Personal de TI, el Personal Administrativo, las Autoridades de la Facultad, el Servicio de Notificaciones, el Sistema de Horario Académico y la infraestructura IoT instalada en las aulas. Posteriormente, el Nivel 1 descompondrá el sistema en procesos específicos como la recopilación de datos ambientales, el monitoreo de equipos, la generación de alertas y la gestión de solicitudes de mantenimiento.

2.5.4. Ventajas y limitaciones de los DFD

A pesar de la amplia adopción de metodologías orientadas a objetos y del Lenguaje Unificado de Modelado (UML), los Diagramas de Flujo de Datos continúan siendo una herramienta útil para representar procesos de negocio y sistemas con fuerte orientación al procesamiento de información. Su principal fortaleza radica en la simplicidad con la que describen la circulación de datos entre procesos, facilitando la comunicación con usuarios no técnicos y apoyando las actividades de validación de requisitos [2, 3].

2.5.4a) Ventajas

Entre sus principales ventajas pueden mencionarse:

- Favorecen la comprensión global del sistema.
- Permiten identificar entradas, salidas y transformaciones de información.
- Facilitan la detección temprana de procesos redundantes o inconsistentes.
- Constituyen un complemento adecuado para los modelos de datos y de comportamiento.

2.5.4b) Limitaciones

No obstante, los DFD presentan algunas limitaciones. Debido a que representan únicamente el flujo de información, no describen aspectos relacionados con la secuencia temporal de eventos, los cambios de estado del sistema o la interacción detallada entre objetos. Por esta razón, suelen emplearse conjuntamente con Diagramas Entidad-Relación, Casos de Uso y Diagramas de Estado, permitiendo obtener una visión integral del sistema analizado [3, 4].

En el proyecto SIGA, por ejemplo, un DFD permitirá visualizar cómo los datos capturados por los sensores IoT son procesados para generar alertas y solicitudes de mantenimiento; sin embargo, el ciclo de vida de una alerta —desde su detección hasta su resolución— será modelado posteriormente mediante un Statechart, mientras que la interacción entre usuarios y funcionalidades específicas será descrita mediante Casos de Uso UML. Esta complementariedad entre modelos constituye uno de los principios fundamentales de la Ingeniería de Requisitos moderna y favorece la construcción de especificaciones completas, consistentes y trazables [1, 2].

2.5.5. Casos de uso y su relación con los requisitos funcionales

Los casos de uso constituyen una de las técnicas más ampliamente utilizadas para modelar los requisitos funcionales de un sistema desde la perspectiva de los usuarios y demás actores externos. Su objetivo principal consiste en describir las interacciones entre un actor y el sistema para alcanzar un resultado de valor, especificando el comportamiento observable del sistema sin detallar los mecanismos internos de implementación. De esta forma, los casos de uso facilitan la comprensión de los servicios que el sistema debe ofrecer y contribuyen a la validación temprana de los requisitos con las partes interesadas [1, 2].

IEEE/ISO/IEC 29148 reconoce los modelos de escenarios y de interacción como mecanismos apropiados para complementar la especificación textual de requisitos, permitiendo representar de manera estructurada el comportamiento esperado del sistema desde la perspectiva de los usuarios. Asimismo, SWEBOK destaca que los casos de uso constituyen una técnica ampliamente adoptada para organizar, documentar y comunicar los requisitos funcionales durante las actividades de análisis y especificación [1, 2].

Un caso de uso describe una secuencia de acciones realizadas por el sistema en respuesta a la interacción con uno o varios actores. Cada caso de uso representa un objetivo específico del negocio y se encuentra asociado con uno o varios requisitos funcionales previamente identificados durante el proceso de elicitación. Esta relación permite establecer una correspondencia directa entre las necesidades de los interesados y las funcionalidades implementadas por el sistema, favoreciendo la trazabilidad a lo largo del ciclo de vida del proyecto [4, 5].

2.5.5a) Elementos de un caso de uso

Desde una perspectiva metodológica, un caso de uso suele estar conformado por los siguientes elementos:

- Nombre del caso de uso.
- Objetivo o propósito.
- Actores participantes.

- Precondiciones.
- Flujo principal de eventos.
- Flujos alternativos o excepcionales.
- Postcondiciones.
- Requisitos funcionales asociados.

Esta estructura permite documentar el comportamiento esperado del sistema de forma comprensible tanto para usuarios finales como para analistas y desarrolladores, reduciendo ambigüedades durante las etapas posteriores del desarrollo.

2.5.6. Actores del sistema SIGA

En el contexto del proyecto SIGA, los actores representan personas u otros sistemas externos que interactúan con el Sistema Inteligente de Gestión de Aulas para intercambiar información o solicitar la ejecución de determinadas funciones. La identificación de estos actores constituye una actividad esencial dentro del análisis funcional, ya que permite delimitar el alcance del sistema y definir claramente sus interfaces externas [1,4].

Considerando el dominio del proyecto, los principales actores identificados son los siguientes:

Cuadro 1: Actores principales del sistema SIGA

Actor	Responsabilidad principal
Personal de TI	Monitorear sensores IoT, administrar equipos tecnológicos y atender incidencias técnicas.
Docente	Consultar el estado del aula y utilizar los dispositivos tecnológicos disponibles durante las actividades académicas.
Personal de Infraestructura y Mantenimiento	Gestionar solicitudes de mantenimiento y registrar las acciones correctivas realizadas.
Personal Administrativo	Consultar indicadores, generar reportes y supervisar el funcionamiento general de la infraestructura.
Autoridades de la Facultad	Visualizar estadísticas e indicadores para apoyar la toma de decisiones.
Sensores y Gateway IoT	Proporcionar datos ambientales y operativos provenientes de la infraestructura inteligente.
Sistema de Horario Académico	Suministrar información relacionada con la programación de aulas.
Servicio de Notificaciones	Enviar alertas y notificaciones a los usuarios correspondientes.
Sistema de Videovigilancia	Compartir información de apoyo para la supervisión de incidentes, sin reemplazar sus funciones propias.

La identificación de estos actores permite definir con precisión las responsabilidades del sistema y evita incorporar funcionalidades fuera del alcance establecido para el proyecto.

2.5.7. Aplicación de los casos de uso al proyecto SIGA

Los casos de uso definidos para SIGA representan las principales funcionalidades identificadas durante el proceso de levantamiento de requisitos. Cada uno de ellos mantiene una relación directa con uno o varios requisitos funcionales documentados en la especificación del sistema, favoreciendo la trazabilidad entre los diferentes artefactos de la

Ingeniería de Requisitos.

Como ejemplo, el requisito funcional RF-08 establece que el sistema debe generar automáticamente alertas cuando los sensores IoT detecten condiciones ambientales fuera de los umbrales establecidos. Este requisito se materializa mediante el Caso de Uso CU-04 *Generar alertas por anomalías*, cuyo flujo principal comprende las siguientes actividades:

1. Recepción de datos provenientes de los sensores IoT.
2. Comparación de las mediciones con los umbrales configurados.
3. Detección de una condición anómala.
4. Registro de la alerta en el sistema.
5. Envío de notificaciones a los usuarios responsables.
6. Inicio del proceso de atención de la incidencia mediante el caso de uso CU-05 *Atender alerta*.

Este ejemplo evidencia cómo un requisito funcional puede representarse mediante un escenario de interacción claramente definido, facilitando tanto su validación como su posterior implementación.

De forma similar, el requisito RF-12 se encuentra asociado con el Caso de Uso CU-06 *Registrar solicitud de mantenimiento*, mediante el cual el Personal de Infraestructura documenta las acciones correctivas derivadas de una alerta previamente generada. Esta relación establece una continuidad lógica entre el monitoreo automático realizado por los sensores IoT y las actividades administrativas necesarias para garantizar la disponibilidad de la infraestructura educativa.

Asimismo, el requisito RF-01 se relaciona con el Caso de Uso CU-01 *Monitorear estado de aula en tiempo real*, mientras que el requisito RF-04 se encuentra asociado al monitoreo y control remoto de proyectores, permitiendo a los responsables de TI supervisar el estado operativo de los dispositivos instalados en las aulas inteligentes.

2.5.8. Relación entre los DFD y los casos de uso

Aunque los Diagramas de Flujo de Datos y los Casos de Uso representan la perspectiva funcional del sistema, ambos modelos responden a objetivos diferentes y se complementan durante el análisis de requisitos. Los DFD describen el movimiento y la transformación de la información entre procesos, mientras que los Casos de Uso enfatizan la interacción entre los actores y el sistema para alcanzar objetivos específicos del negocio [2, 3].

En el proyecto SIGA, por ejemplo, el proceso de generación automática de alertas puede representarse mediante un DFD que describa el flujo de datos desde los sensores IoT hasta el módulo de notificaciones. Paralelamente, el Caso de Uso CU-04 permite

documentar la interacción entre el sistema y el Personal de TI durante la gestión de dichas alertas. Ambos modelos representan la misma funcionalidad desde perspectivas diferentes y complementarias, fortaleciendo la consistencia de la especificación de requisitos.

Esta complementariedad favorece además la trazabilidad entre los distintos artefactos del proyecto. Un requisito funcional documentado en el SRS puede localizarse posteriormente en un caso de uso, en un proceso del DFD y, más adelante, en los modelos de comportamiento y diseño, garantizando que todas las funcionalidades identificadas durante el análisis permanezcan alineadas durante el desarrollo del sistema, conforme a las prácticas recomendadas por IEEE/ISO/IEC 29148 y SWEBOK [1, 2].

Como consecuencia, los modelos funcionales no deben considerarse artefactos independientes, sino componentes complementarios de una especificación integral de requisitos. Mientras los Diagramas de Flujo de Datos describen cómo circula la información dentro del sistema, los Casos de Uso muestran cómo los diferentes actores interactúan con dicho sistema para alcanzar objetivos concretos. En la siguiente sección se abordará la perspectiva de comportamiento, la cual permitirá modelar la evolución dinámica de los estados del sistema y complementar las perspectivas de datos y funcional previamente desarrolladas.

2.6. Modelos de comportamiento (Perspectiva de comportamiento)

2.6.1. Concepto y propósito de los modelos de comportamiento

La perspectiva de comportamiento describe la evolución dinámica de un sistema en respuesta a eventos internos o externos, permitiendo representar los cambios de estado que experimentan sus componentes durante la ejecución de los procesos definidos en la especificación de requisitos. A diferencia de los modelos de datos, que representan la estructura de la información, y de los modelos funcionales, que describen el procesamiento de dicha información, los modelos de comportamiento muestran la secuencia de estados, eventos, condiciones y transiciones que caracterizan el funcionamiento del sistema a lo largo del tiempo [1, 2].

Dentro de la Ingeniería de Requisitos, estos modelos desempeñan un papel fundamental al facilitar la comprensión del comportamiento esperado del sistema frente a diferentes escenarios operacionales. IEEE/ISO/IEC 29148 establece que la especificación de requisitos puede complementarse mediante modelos de comportamiento con el propósito de reducir ambigüedades, mejorar la comunicación con las partes interesadas y facilitar las actividades posteriores de diseño, implementación y validación del software [1]. Asimismo, SWEBOK destaca que la utilización de modelos dinámicos contribuye a verificar la consistencia entre los requisitos funcionales y la respuesta esperada del sistema frente a distintos eventos del entorno [2].

En sistemas inteligentes como SIGA, cuyo funcionamiento depende de la interacción continua entre sensores IoT, dispositivos electrónicos, usuarios y servicios externos, la representación del comportamiento resulta especialmente importante. Eventos como la detección de anomalías ambientales, la activación de alertas, el control remoto de equipos tecnológicos o la gestión de solicitudes de mantenimiento implican cambios sucesivos de estado que no pueden describirse adecuadamente únicamente mediante modelos funcionales o estructurales.

2.6.2. Statecharts como modelo de comportamiento

Uno de los modelos más influyentes para representar el comportamiento dinámico de sistemas complejos es el *Statechart*, propuesto por David Harel en 1987 como una extensión de las máquinas de estados finitos tradicionales. Su principal objetivo consiste en superar las limitaciones de los diagramas de estados clásicos mediante la incorporación de mecanismos de jerarquía, concurrencia y comunicación entre estados, permitiendo modelar sistemas con un elevado grado de complejidad de forma clara y estructurada [6].

A diferencia de una máquina de estados convencional, un Statechart no solamente representa estados y transiciones, sino que también introduce conceptos que permiten organizar el comportamiento del sistema en diferentes niveles de abstracción. Esta capacidad resulta especialmente útil en sistemas distribuidos o basados en eventos, donde múltiples procesos pueden ejecutarse simultáneamente y responder de manera independiente a diferentes estímulos.

Desde esta perspectiva, los Statecharts constituyen uno de los antecedentes conceptuales de los actuales Diagramas de Estados definidos por UML, conservando los principios fundamentales propuestos por Harel y ampliamente utilizados en la Ingeniería de Software contemporánea [2, 4].

2.6.3. Elementos fundamentales de un Statechart

Según Harel, un Statechart está conformado por diversos elementos que permiten representar de manera precisa el comportamiento dinámico de un sistema [6].

2.6.3a) Estados

Los estados representan las diferentes situaciones en las que puede encontrarse un objeto durante su ciclo de vida. Cada estado describe una condición estable del sistema mientras no ocurra un evento que provoque una transición hacia otro estado.

En el contexto del sistema SIGA, una entidad como *Alerta* puede evolucionar desde su creación hasta su resolución atravesando diferentes estados operacionales.

2.6.3b) Transiciones

Las transiciones representan los cambios de un estado a otro como consecuencia de la ocurrencia de un determinado evento. Cada transición puede estar asociada a una acción específica que será ejecutada durante el cambio de estado.

Por ejemplo, la recepción de una medición fuera de los umbrales establecidos por un sensor IoT puede provocar la transición desde un estado de monitoreo normal hacia un estado de alerta detectada.

2.6.3c) Eventos

Los eventos constituyen los estímulos que desencadenan una transición entre estados. Estos pueden originarse por acciones realizadas por los usuarios, por procesos internos del sistema o por dispositivos externos.

En SIGA, algunos ejemplos de eventos son:

- Recepción de una nueva lectura del sensor.
- Detección de temperatura fuera del rango permitido.
- Confirmación de recepción de la alerta.
- Finalización de una intervención de mantenimiento.

2.6.3d) Condiciones de guarda (Guard Conditions)

Las condiciones de guarda corresponden a expresiones lógicas que deben cumplirse para que una transición pueda ejecutarse. Estas permiten modelar decisiones dependientes del contexto y representan uno de los mecanismos más importantes para controlar el comportamiento del sistema.

Por ejemplo, una transición puede definirse mediante la condición:

`temperatura >umbralMaximo`

de manera que únicamente cuando dicha condición sea verdadera el sistema genere una nueva alerta.

2.6.3e) Acciones

Las acciones representan operaciones ejecutadas como consecuencia de una transición o durante la permanencia en un determinado estado.

Dentro de SIGA, una acción típica consiste en registrar automáticamente la alerta en la base de datos y enviar una notificación al Personal de TI inmediatamente después de detectar una condición anómala.

2.6.4. Jerarquía y concurrencia

Una de las principales contribuciones de Harel consiste en la incorporación de estados jerárquicos y estados concurrentes, características ausentes en los diagramas de estados tradicionales.

Los estados jerárquicos permiten agrupar múltiples subestados relacionados dentro de un estado compuesto, reduciendo significativamente la complejidad del modelo cuando el sistema posee numerosos estados intermedios.

Por otra parte, la concurrencia permite representar actividades que ocurren de manera simultánea. En lugar de describir múltiples máquinas de estados independientes, un único Statechart puede modelar regiones concurrentes que evolucionan paralelamente bajo determinadas condiciones.

Esta capacidad resulta particularmente relevante para sistemas IoT como SIGA, donde el monitoreo ambiental y la supervisión de conectividad pueden ejecutarse simultáneamente mientras un aula permanece ocupada. En consecuencia, un estado compuesto denominado *Aula Ocupada* puede dividirse en dos regiones concurrentes: una dedicada al monitoreo ambiental y otra al monitoreo de la infraestructura tecnológica. Ambas evolucionan de forma independiente, pero comparten el mismo contexto operacional, proporcionando una representación más fiel del comportamiento real del sistema [4, 6].

2.6.5. Ventajas de los Statecharts frente a los diagramas de estados tradicionales

Los Statecharts representan una evolución significativa respecto a las máquinas de estados finitas clásicas. Su utilización permite modelar sistemas de gran complejidad sin incrementar excesivamente el número de estados ni de transiciones, favoreciendo la modularidad y la reutilización de modelos.

Entre sus principales ventajas destacan:

- Incorporación de jerarquía entre estados.
- Representación de regiones concurrentes.
- Soporte para eventos asíncronos.
- Modelado de condiciones de guarda.
- Mayor escalabilidad para sistemas complejos.
- Correspondencia directa con los Diagramas de Estado UML.

Estas características explican por qué los principios propuestos por Harel continúan siendo utilizados en herramientas modernas de modelado UML y en numerosos procesos

de Ingeniería de Requisitos orientados al desarrollo de sistemas distribuidos, ciberfísicos e inteligentes [2, 4, 6].

2.6.6. Diagramas de secuencia como complemento de los modelos de comportamiento

Aunque los Statecharts permiten representar la evolución de estados de un objeto a lo largo de su ciclo de vida, existen escenarios en los cuales resulta necesario describir la interacción temporal entre múltiples participantes del sistema. Para este propósito, el Lenguaje Unificado de Modelado (UML) incorpora los Diagramas de Secuencia, los cuales muestran el intercambio de mensajes entre actores, objetos o componentes siguiendo un orden cronológico [2, 4].

Los Diagramas de Secuencia complementan los modelos funcionales y los Statecharts al proporcionar una representación detallada de cómo colaboran los distintos elementos del sistema para ejecutar una funcionalidad determinada. En este tipo de diagramas, cada participante se representa mediante una línea de vida (*lifeline*), mientras que los mensajes intercambiados indican la invocación de operaciones, el envío de información o la respuesta generada por otro componente del sistema.

Dentro de la Ingeniería de Requisitos, estos diagramas permiten validar que las interacciones descritas en los casos de uso sean consistentes con el comportamiento esperado del sistema, facilitando además la identificación de responsabilidades entre componentes antes de iniciar el diseño detallado del software [1, 2].

En el proyecto SIGA, un diagrama de secuencia puede emplearse para representar la interacción entre los sensores IoT, el módulo de monitoreo, el servicio de generación de alertas, el servicio de notificaciones y el Personal de TI durante la detección de una anomalía ambiental. De esta forma, se obtiene una visión dinámica del flujo de mensajes que complementa la información representada mediante los Diagramas de Flujo de Datos y los Statecharts.

2.6.7. Aplicación de los Statecharts al sistema SIGA

Con el propósito de ilustrar la aplicación de los modelos de comportamiento, se considera el ciclo de vida de la entidad *Alerta*, la cual representa uno de los elementos centrales del sistema SIGA debido a su relación directa con el monitoreo de condiciones ambientales y el mantenimiento preventivo de la infraestructura.

Durante su funcionamiento, una alerta atraviesa diferentes estados desde el momento en que es detectada hasta su resolución definitiva. Cada transición depende de eventos específicos y, en determinados casos, de condiciones de guarda que verifican el cumplimiento de determinadas reglas de negocio.

La Figura 1 presenta un Statechart conceptual correspondiente al ciclo de vida de una alerta.

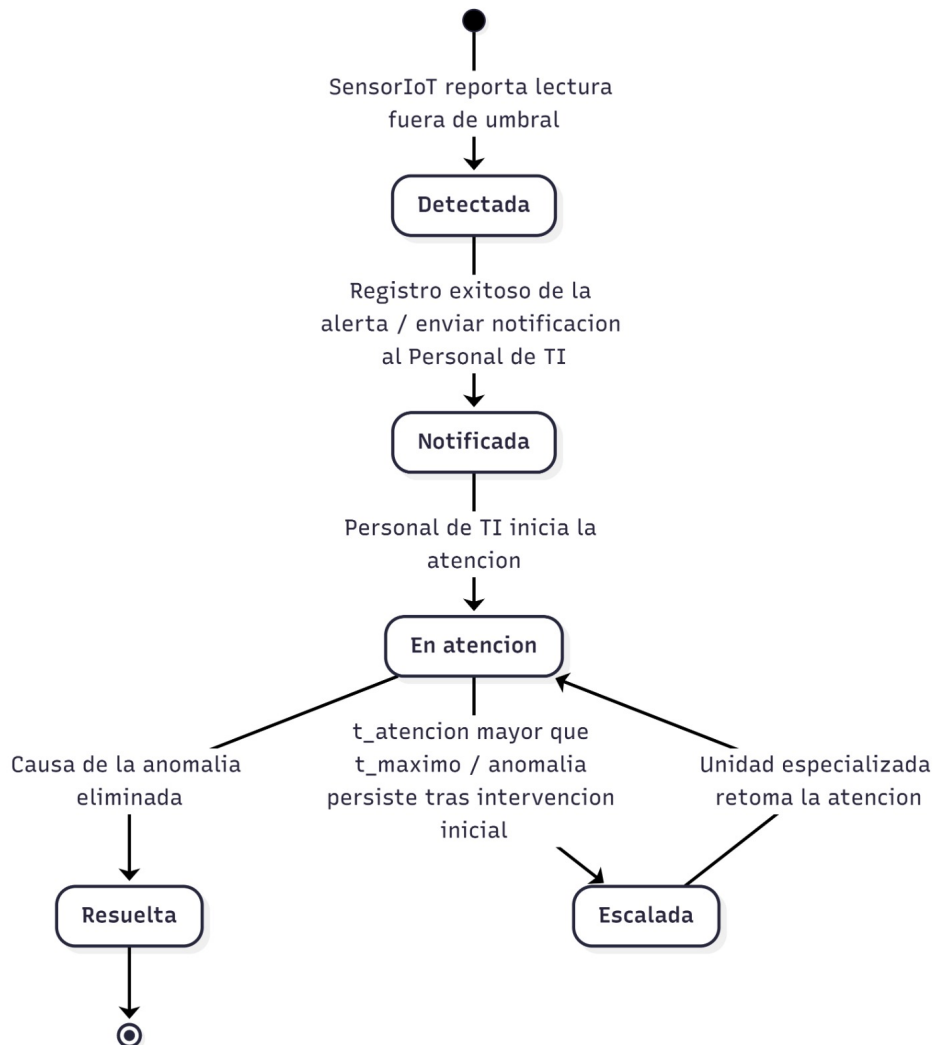


Figura 1: Statechart conceptual del ciclo de vida de una alerta en el sistema SIGA.

El modelo considera los siguientes estados principales:

- **Detectada:** el sistema identifica una condición fuera de los umbrales establecidos mediante las lecturas provenientes de los sensores IoT.
- **Notificada:** la alerta ha sido registrada y enviada al Personal de TI mediante el servicio de notificaciones.
- **En atención:** el incidente se encuentra siendo evaluado por el personal responsable.
- **Resuelta:** la causa de la anomalía ha sido eliminada y la alerta es cerrada.
- **Escalada:** la incidencia requiere intervención especializada o atención por parte de otra unidad administrativa.

Las transiciones entre estos estados se encuentran condicionadas por diferentes eventos del sistema. Por ejemplo, la transición entre los estados *Detectada* y *Notificada* ocurre inmediatamente después del registro exitoso de la alerta, mientras que la transición hacia el estado *Escalada* únicamente se ejecuta cuando el tiempo máximo de atención supera el umbral establecido o cuando la anomalía persiste después de la intervención inicial.

Una posible condición de guarda puede expresarse de la siguiente manera:

$$[t_{atencion} > t_{maximo}]$$

donde t_{maximo} representa el tiempo máximo permitido para la atención de una alerta antes de iniciar el procedimiento de escalamiento.

Este tipo de condiciones permite modelar explícitamente las reglas de negocio que gobiernan el comportamiento del sistema y facilitan posteriormente las actividades de validación y prueba de los requisitos.

2.6.8. Estado compuesto y concurrencia en el monitoreo de aulas

Una de las principales ventajas de los Statecharts consiste en la posibilidad de representar comportamientos concurrentes mediante estados compuestos. Esta característica resulta especialmente útil en sistemas IoT, donde múltiples procesos pueden ejecutarse simultáneamente sin interferir entre sí.

En el proyecto SIGA, el estado *Aula Ocupada* puede modelarse como un estado compuesto que contiene dos regiones concurrentes:

1. Monitoreo ambiental.
2. Monitoreo de conectividad y dispositivos tecnológicos.

Mientras la primera región supervisa continuamente variables como temperatura, humedad y calidad del aire, la segunda verifica el estado operativo de proyectores, climatizadores, gateways IoT y demás dispositivos instalados dentro del aula.

Ambas regiones evolucionan de manera independiente, pero permanecen sincronizadas dentro del estado compuesto. Si cualquiera de ellas detecta una condición anómala, el sistema genera una nueva alerta y ejecuta las acciones correspondientes para notificar al personal responsable.

Este mecanismo permite representar de forma precisa el comportamiento paralelo del sistema, evitando la proliferación innecesaria de estados y transiciones que caracterizaría a una máquina de estados convencional [6].

2.6.9. Relación entre modelos de comportamiento y requisitos funcionales

Los modelos de comportamiento mantienen una relación directa con los requisitos funcionales documentados durante la especificación del sistema. Cada transición representada

en un Statechart o en un Diagrama de Secuencia debe corresponder a una funcionalidad previamente identificada y validada durante la Ingeniería de Requisitos.

En el caso del proyecto SIGA, el requisito funcional RF-08 establece la generación automática de alertas cuando se detectan condiciones ambientales anómalas. Dicho requisito se refleja directamente en el Statechart mediante la transición desde el estado inicial hacia el estado *Detectada*. Posteriormente, el requisito RF-12 se materializa cuando una alerta atendida genera una solicitud de mantenimiento que inicia el proceso administrativo correspondiente.

Esta correspondencia favorece la trazabilidad entre los distintos artefactos del proyecto, permitiendo verificar que cada requisito funcional tenga representación tanto en los modelos funcionales como en los modelos de comportamiento. Asimismo, facilita las actividades de validación, verificación y pruebas del sistema durante las fases posteriores del ciclo de vida, conforme a las recomendaciones establecidas por IEEE/ISO/IEC 29148 y Pohl [1, 4].

En consecuencia, los modelos de comportamiento complementan las perspectivas de datos y funcional previamente desarrolladas, proporcionando una visión dinámica del sistema que resulta indispensable para comprender la evolución de sus componentes frente a eventos internos y externos. La integración de estas tres perspectivas constituye uno de los principios fundamentales de la Ingeniería de Requisitos y será abordada en la siguiente sección mediante el análisis de la trazabilidad y la consistencia entre modelos.

2.7. Interrelaciones entre las perspectivas de datos, funcional y de comportamiento

2.7.1. Concepto de integración entre las tres perspectivas

La Ingeniería de Requisitos no se limita a la identificación y documentación de necesidades de los interesados, sino que también establece mecanismos para garantizar la consistencia entre los diferentes modelos utilizados durante la especificación del sistema. En este contexto, las perspectivas de datos, funcional y de comportamiento constituyen representaciones complementarias de un mismo dominio, cada una orientada a describir un aspecto particular del sistema. Su integración permite construir especificaciones completas, reducir inconsistencias y facilitar la trazabilidad de los requisitos durante todo el ciclo de vida del proyecto [1, 7].

IEEE/ISO/IEC 29148 establece que la especificación de requisitos debe mantener relaciones explícitas entre los diferentes artefactos producidos durante la Ingeniería de Requisitos, permitiendo verificar que cada necesidad identificada por los interesados pueda rastrearse hasta los elementos de diseño, implementación, validación y mantenimiento correspondientes. De forma complementaria, ISO/IEC/IEEE 15288 considera la trazabilidad como un mecanismo esencial para mantener la coherencia entre los procesos del ciclo

de vida del sistema y los productos de trabajo generados en cada una de sus etapas [1, 7].

En consecuencia, los modelos conceptuales desarrollados durante el análisis no deben interpretarse como documentos independientes, sino como representaciones coordinadas que describen diferentes perspectivas del mismo sistema. Mientras el modelo de datos representa la información administrada por el sistema, el modelo funcional describe la transformación de dicha información y el modelo de comportamiento explica la evolución temporal de sus componentes frente a distintos eventos.

2.7.2. Trazabilidad entre SyRS y SRS

Uno de los principios fundamentales establecidos por IEEE/ISO/IEC 29148 consiste en garantizar la trazabilidad bidireccional entre los requisitos del sistema (System Requirements Specification, SyRS) y los requisitos del software (Software Requirements Specification, SRS). Esta relación permite demostrar que cada requisito de software responde a una necesidad previamente identificada a nivel del sistema y que todas las funcionalidades implementadas contribuyen al cumplimiento de los objetivos definidos por las partes interesadas [1].

En el proyecto SIGA, esta trazabilidad puede observarse claramente en el requisito funcional RF-08, correspondiente a la generación automática de alertas por anomalías ambientales. Desde la perspectiva del sistema, el SyRS establece la necesidad de supervisar continuamente las condiciones de las aulas y notificar oportunamente situaciones que puedan afectar la operación de la infraestructura educativa. A partir de esta necesidad se derivan requisitos específicos del software relacionados con la captura de datos provenientes de sensores IoT, el análisis de las mediciones recibidas, la generación de alertas y el envío de notificaciones a los responsables correspondientes.

Este refinamiento progresivo permite mantener una relación explícita entre las necesidades del negocio, las funcionalidades implementadas y los mecanismos de validación definidos durante el desarrollo del sistema.

2.7.3. Integración de las tres perspectivas

La integración entre los modelos de datos, funcional y de comportamiento puede ilustrarse mediante el requisito funcional RF-08 del sistema SIGA.

Desde la perspectiva de datos, dicho requisito se encuentra representado mediante la entidad *Alerta*, la cual almacena información relacionada con el tipo de anomalía detectada, su prioridad, el momento de generación, el estado actual y la relación con la correspondiente solicitud de mantenimiento.

Desde la perspectiva funcional, el mismo requisito aparece representado mediante el proceso de generación automática de alertas dentro del Diagrama de Flujo de Datos y mediante el Caso de Uso CU-04 *Generar alertas por anomalías*, en el cual participan los

sensores IoT, el servicio de monitoreo y el Personal de TI.

Finalmente, desde la perspectiva de comportamiento, el requisito se materializa mediante el Statechart correspondiente al ciclo de vida de la entidad *Alerta*, describiendo la transición entre los estados *Detectada*, *Notificada*, *En atención*, *Resuelta* y *Escalada* en función de los eventos generados durante la operación del sistema.

Esta correspondencia demuestra que un mismo requisito funcional puede representarse de manera consistente mediante distintos modelos, cada uno orientado a describir un aspecto específico del sistema sin introducir contradicciones entre ellos. Como resultado, se obtiene una especificación más completa y fácilmente verificable durante las etapas posteriores del ciclo de vida del software [1, 4].

2.7.4. Consistencia entre modelos

Además de la trazabilidad, la Ingeniería de Requisitos requiere garantizar la consistencia entre los diferentes artefactos utilizados durante el análisis. SWEBOK señala que la verificación de consistencia constituye una actividad permanente cuyo propósito consiste en identificar contradicciones, redundancias u omisiones antes de iniciar el diseño y la implementación del sistema [2].

Pohl complementa este enfoque indicando que la consistencia debe verificarse tanto de forma horizontal como vertical. La consistencia horizontal evalúa la correspondencia entre modelos desarrollados dentro del mismo nivel de abstracción, mientras que la consistencia vertical verifica que los requisitos refinados mantengan alineación con las necesidades identificadas en niveles superiores de especificación [4].

En el caso del proyecto SIGA, la consistencia horizontal puede verificarse comprobando que las entidades utilizadas en el modelo Entidad–Relación correspondan a los almacenes de datos representados en los Diagramas de Flujo de Datos y a los objetos participantes en los Diagramas de Secuencia. De igual manera, el estado inicial de la entidad *Alerta* representado en el Statechart debe corresponder al proceso de generación de alertas definido en el modelo funcional.

Por otra parte, la consistencia vertical requiere verificar que cada requisito funcional representado mediante un caso de uso pueda rastrearse hasta una necesidad previamente identificada en el SyRS y que, posteriormente, dicho requisito pueda asociarse con componentes específicos del diseño, casos de prueba y criterios de aceptación definidos durante el desarrollo del proyecto.

Referencias

- [1] *ISO/IEC/IEEE 29148:2018 Systems and software engineering — Life cycle processes — Requirements engineering*, International Organization for Standardization (ISO)

- and International Electrotechnical Commission (IEC) and Institute of Electrical and Electronics Engineers (IEEE) Std. ISO/IEC/IEEE 29148:2018, 2018, requirements Engineering Standard. [Online]. Available: <https://www.iso.org/standard/72089.html>
- [2] *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide) Version 4.0*, 4th ed. Los Alamitos, California, USA: IEEE Computer Society, 2024. [Online]. Available: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- [3] L. A. Maciaszek, *Requirements Analysis and System Design: Developing Information Systems with UML*, 1st ed. Harlow, England: Addison-Wesley, 2007.
- [4] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*, 2nd ed. Cham, Switzerland: Springer, 2025, part IV: Requirements Artefacts. [Online]. Available: <https://link.springer.com/book/9783662692042>
- [5] *Certified Professional for Requirements Engineering (CPRE) Foundation Level Syllabus*, 3rd ed., International Requirements Engineering Board (IREB), Nuremberg, Germany, 2025, foundation Level Syllabus. [Online]. Available: <https://cpre.ireb.org>
- [6] D. Harel, “Statecharts: A visual formalism for complex systems,” *Science of Computer Programming*, vol. 8, no. 3, pp. 231–274, 1987.
- [7] *ISO/IEC/IEEE 15288:2023 Systems and software engineering — System life cycle processes*, International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), and Institute of Electrical and Electronics Engineers (IEEE) Std. ISO/IEC/IEEE 15288:2023, 2023, system Life Cycle Processes. [Online]. Available: <https://www.iso.org/standard/81702.html>